Lab 03 – (Week of September 9th)

All of the activities of this lab are to be completed in teams of two (or three, but only as necessary), and each team is limited to the use of one computer. The activities involving use of the "Practice-It" website are to be done through just one of the student authorizations.

Activity 1 – Practice-It Self-Checks

At the Practice-It web site, http://practiceit.cs.washington.edu/, log in and then click the **Start Practicing** button. Then click on the link labeled **Building Java Programs, 4th Edition** and then on the link indicating **BJP4 Chapter 2**.

Do the following **Practice-It Self-Checks** and, when finished, show your GTA (graduate teaching assistant) that you've completed all of them correctly. (Go back to the web page that lists all the Chapter 2 Self-Checks and refresh it. All that were answered correctly will be indicated on the page.)

Several of these questions involve evaluating one or more arithmetic expressions. Keep in mind the rules that govern the order in which operators are applied, which are explained in the Reges & Stepp textbook. Unary operators (as in `-13` or `+9`) have highest precedence, followed by the (binary) multiplicative operators (`*`, `/`, and `%`), followed by the (binary) additive operators (`+` and `-`). Consecutive operators of the same precedence are applied from left to right.

Also keep in mind that if both operands of a division operator (`/`) are of type `int`, the result is also of type `int`.

1. **BJP4 Self-Check 2.1 (legalIntLiterals)**: Well-formed literals.
2. **BJP Self-Check 2.2 (evaluatingExpressions)**: Evaluate the given expression.
3. **BJP Self-Check 2.3 (expressions1)**: Evaluate each of the specific expressions, as identified by your instructor.
4. **BJP Self-Check 2.4 (expressions2)**: Evaluate each of the specific expressions, as identified by your instructor.
5. **BJP Self-Check 2.5 (expressions3)**: Evaluate each of the specific expressions, as identified by your instructor. Keep in mind that the plus operator (+) refers to String concatenation, rather than addition, if either of its operands is of type `String`. In that case, the non-String operand is "converted" to a value of type `String` (i.e. a sequence of characters) before the operation is carried out.
6. **BJP Self-Check 2.6 (declareRealNumberSyntax)**: Identify the line of code that declares and initializes a variable as specified.
7. **BJP Self-Check 2.7 (fitnessVariables)**: Read the description carefully to determine the most appropriate data type to use for each variable.
8. **BJP Self-Check 2.8 (studentVariables)**: Again, read the description carefully to determine the most appropriate data type to use for each variable.
9. **BJP Self-Check 2.9 (lastDigitExpression)**: It's odd that the instructions say to use "division and modulus" when only the latter is needed.
10. **BJP Self-Check 2.10 (Oops2-errors)**: Correct the (mostly) syntax errors in the given Java program.
11. **BJP Self-Check 2.11 (digitExpressions)**: This is similar to Self-Check 2.9, but here you **do** need to use **both** division (/) and modulus (%).
12. **BJP Self-Check 2.12 (valueOfX)**: Given a sequence of assignments to variable x, what is its value immediately thereafter?

13. **BJP Self-Check 2.14 (firstSecond)**: Given a sequence of assignments to variables first and second, what are their values immediately thereafter?
14. **BJP Self-Check 2.17 (maxMin)**: Treat these as a sequence of instructions.
15. **BJP Self-Check 2.19 (ComputePay)**:

Remember, before moving on to the next activity show your GTA that you've correctly answered all the Self-Checks assigned above.

Activity 2 – Lengths (Implementation)

This activity involves the use of methods and is meant to give you practice in formulating arithmetic expressions. Note that this program makes use of "global" variables; that is, static variables declared within the scope of a class. This is generally frowned upon, but it is something that we will soon be able to avoid, after we've learned about arguments/parameters for methods and how to make a method return a value to its caller.

The given file, `Lengths.java`, is an incomplete, yet compilable and executable, version of a Java application. Comments that say "`CODE MISSING HERE!`" indicate where instructions should be inserted in order to get the program working as intended.

The program is intended to prompt the user to enter a positive integer representing a length, measured in inches. After reading the user's response, the program displays two alternative ways of expressing that same length. To help you better visualize this intention, consider the following three example uses of this program.

```
c:\C134L\Lab03\>java Lengths
Enter length in inches>45
45 inches is equivalent to:
3.75 feet and also
3 full feet and 9 residual inches.

c:\C134L\Lab03\>java Lengths
Enter length in inches>1000
1000 inches is equivalent to:
83.33333333333333 feet and also
83 full feet and 4 residual inches.

c:\C134L\Lab03\>java Lengths
Enter length in inches>3600
3600 inches is equivalent to:
300.0 feet and also
300 full feet and 0 residual inches.
```

Once complete, show your GTA that this program compiles and executes correctly in jGRASP. Be sure to complete the comments at the top of the source program, being sure to provide the complete names of all team members. Recompile and rerun, just to be certain, and then submit this revised file to the CMPS 134L CWS for the section you are working in.

Activity 3: Practice-It Self-Checks and Exercises on for-loops

After logging in to Practice-It, click on the link labeled **Building Java Programs, 4th Edition** and then on the link indicating **BJP Chapter 2**.

Do the following Self-Checks and, when finished, show your GTA (graduate teaching assistant) that you've completed all of them correctly. (Go back to the web page that lists all the Chapter 2 Self-Checks and refresh it. Any that were answered correctly will be indicated on the page.)

For the questions that ask you to provide the output for some given source code, do not merely run the program and have the computer do the work; figure it out yourselves first. Little will be gained otherwise.

1. **BJP4 Self-Check 2.21(Count2)**: Complete the given source code, replacing the "FINISH ME" parts with your own code, to produce the given output.
2. **BJP4 Self-Check 2.25(numberTotal):** Show output produced by the execution of a given Java code segment involving a for-loop.
3. **BJP4 Self-Check 2.26 (fingerTrap)**: Show output produced by the execution of a given Java code segment involving a for-loop. Recall that **** is an "escape sequence" that evaluates to the character ****.
4. **BJP4 Self-Check 2.28 (blastOff)**: Show output produced by the execution of a given Java code segment involving a for-loop. The notation `i--` is a shorthand way of saying `i = i - 1`.
5. **BJP4 Self-Check 2.29 (multiplication)**: Show output produced by the execution of a given Java for-loop (that has a loop nested inside it). The notation `j++` is a shorthand way of saying `j = j + 1`. Don't forget the difference between the behaviors of `System.out.print()` and `System.out.println()`
6. **BJP4 Self-Check 2.30 (nestedLoops)**: (optional) Show output produced by the execution of a given Java program. The figure that it "draws" should be familiar to you from our recent lecture. The ++ operator is as mentioned above.
7. **BJP4 Self-Check 2.31 (starExclamation1)**: Show output produced by the execution of a given Java code segment involving for-loops nested three levels deep.
8. **BJP4 Self-Check 2.32 (starExclamation2)**: Same as previous problem, except for the placement of the statement that prints an exclamation point.
9. **BJP4 Self-Check 2.33 (starExclamation3)**: Same as previous problem, again except for the placement of the statement that prints an exclamation point.
10. **BJP4 Exercise 2.2: (loopSquares):** Write a for loop that produces the specified output.
11. **BJP4 Exercise 2.3: (fibonacci):** Write a for loop that produces the specified output.
12. **BJP4 Exercise 2.4: (starSquare):** Develop for loops to produce a square-like image.
13. **BJP4 Exercise 2.5: (starTriangle):** Develop for loops to produce a triangle-like image.
14. **BJP4 Exercise 2.6: (numberTriangle):** Develop for loops to produce a triangle-like image of numbers.